🎓 UNIME Python — Exam Exercises

Prof. Fiumara style · medium → hard · 32 problems with numeric data · tap **Show solution** to reveal full code + answer

🔗 Doubly Linked List — 8 exercises

1 MEDIUM

Sum of a doubly linked list

Write dll_sum(dll) that returns the sum of all node values, then give the result for this DLL.

GIVEN

```
dll = {
    'head': {'prev': None, 'next': 7},
    7:      {'prev': 'head', 'next': 4},
    4:      {'prev': 7, 'next': 10},
    10:     {'prev': 4, 'next': 2},
    2:      {'prev': 10, 'next': 'tail'},
    'tail': {'prev': 2, 'next': None}
}
```

FULL SOLUTION

```
def dll_sum(dll):
    current = dll['head']['next']
    total = 0
    while current != 'tail':
        total += current
        current = dll[current]['next']
    return total
```

Answer: 7 + 4 + 10 + 2 = 23

2 MEDIUM

Maximum value in a DLL

Write dll_max(dll) returning the largest node value. Give the answer for this DLL.

GIVEN

```
dll = {
    'head': {'prev': None, 'next': 3},
    3:      {'prev': 'head', 'next': 15},
    15:     {'prev': 3, 'next': 9},
    9:      {'prev': 15, 'next': 21},
    21:     {'prev': 9, 'next': 6},
    6:      {'prev': 21, 'next': 'tail'},
    'tail': {'prev': 6, 'next': None}
}
```

FULL SOLUTION

```
def dll_max(dll):
    current = dll['head']['next']
    max_val = current
    while current != 'tail':
        if current > max_val:
            max_val = current
        current = dll[current]['next']
    return max_val
```

Answer: max(3,15,9,21,6) = 21

3 MEDIUM

Count even values

Write count_even_dll(dll) that returns how many node values are even. Answer for this DLL.

GIVEN

```
dll = {
    'head': {'prev': None, 'next': 8},
    8:      {'prev': 'head', 'next': 5},
    5:      {'prev': 8, 'next': 12},
    12:     {'prev': 5, 'next': 7},
    7:      {'prev': 12, 'next': 4},
    4:      {'prev': 7, 'next': 'tail'},
    'tail': {'prev': 4, 'next': None}
}
```

FULL SOLUTION

```
def count_even_dll(dll):
    current = dll['head']['next']
    count = 0
    while current != 'tail':
        if current % 2 == 0:
            count += 1
        current = dll[current]['next']
    return count
```

Answer: even values = 8, 12, 4 → 3

4 HARD

Is the DLL sorted ascending?

Write dll_sorted(dll) returning True if values are in ascending order. Answer for this DLL.

GIVEN

```
dll = {
    'head': {'prev': None, 'next': 2},
    2:      {'prev': 'head', 'next': 5},
    5:      {'prev': 2, 'next': 9},
    9:      {'prev': 5, 'next': 6},
    6:      {'prev': 9, 'next': 'tail'},
    'tail': {'prev': 6, 'next': None}
}
```

FULL SOLUTION

```
def dll_sorted(dll):
    current = dll['head']['next']
    while dll[current]['next'] != 'tail':
        nxt = dll[current]['next']
        if current > nxt:
            return False
        current = nxt
    return True
```

Answer: 2≤5≤9 but 9 > 6 → False

5 HARD

Enqueue then dequeue (trace the result)

enqueue adds at the tail, dequeue removes from the head. Starting from the DLL below, perform enqueue(dll, 50) then dequeue(dll). Write both functions and give the final order of values (head → tail).

GIVEN

```
dll = {
    'head': {'prev': None, 'next': 10},
    10:     {'prev': 'head', 'next': 20},
    20:     {'prev': 10, 'next': 30},
    30:     {'prev': 20, 'next': 'tail'},
    'tail': {'prev': 30, 'next': None}
}
```

FULL SOLUTION

```
def enqueue(dll, x):
    prev = dll['tail']['prev']
    dll[x] = {'prev': prev, 'next': 'tail'}
    dll[prev]['next'] = x
    dll['tail']['prev'] = x
    return dll

def dequeue(dll):
    first = dll['head']['next']
    right = dll[first]['next']
    dll['head']['next'] = right
    dll[right]['prev'] = 'head'
    del dll[first]
    return dll
```

Answer: After enqueue(50): 10,20,30,50. After dequeue (removes 10): **20 → 30 → 50**

6 HARD

Remove the last two nodes

Write remove_last2(dll) that deletes the final two nodes. Give the resulting order (head → tail).

GIVEN

```
dll = {
    'head': {'prev': None, 'next': 11},
    11:     {'prev': 'head', 'next': 22},
    22:     {'prev': 11, 'next': 33},
    33:     {'prev': 22, 'next': 44},
    44:     {'prev': 33, 'next': 'tail'},
    'tail': {'prev': 44, 'next': None}
}
```

FULL SOLUTION

```
def remove_last2(dll):
    n1 = dll['tail']['prev']      # 44
    n2 = dll[n1]['prev']          # 33
    left = dll[n2]['prev']        # 22
    dll[left]['next'] = 'tail'
    dll['tail']['prev'] = left
    del dll[n1]
    del dll[n2]
    return dll
```

Answer: Removes 44 and 33 → 11 → 22

7 HARD

Swap first and last node

Write swap_ends(dll) that swaps the first and last real nodes. Give the resulting order (head → tail).

GIVEN

```
dll = {
    'head': {'prev': None, 'next': 1},
    1:      {'prev': 'head', 'next': 2},
    2:      {'prev': 1, 'next': 3},
    3:      {'prev': 2, 'next': 4},
    4:      {'prev': 3, 'next': 'tail'},
    'tail': {'prev': 4, 'next': None}
}
```

FULL SOLUTION

```
def swap_ends(dll):
    first = dll['head']['next']   # 1
    last = dll['tail']['prev']    # 4
    if first == last:
        return dll
    fn = dll[first]['next']       # 2
    lp = dll[last]['prev']        # 3
    dll['head']['next'] = last
    dll[last]['prev'] = 'head'
    dll[last]['next'] = fn
    dll[fn]['prev'] = last
    dll[lp]['next'] = first
    dll[first]['prev'] = lp
    dll[first]['next'] = 'tail'
    dll['tail']['prev'] = first
    return dll
```

Answer: Swap 1 and 4 → 4 → 2 → 3 → 1

8 MEDIUM

Count values greater than n

Write count_greater_dll(dll, n) returning how many values exceed n. Answer for n = 10.

GIVEN

```
dll = {
    'head': {'prev': None, 'next': 14},
    14:     {'prev': 'head', 'next': 9},
    9:      {'prev': 14, 'next': 25},
    25:     {'prev': 9, 'next': 3},
    3:      {'prev': 25, 'next': 11},
    11:     {'prev': 3, 'next': 'tail'},
    'tail': {'prev': 11, 'next': None}
}
n = 10
```

FULL SOLUTION

```
def count_greater_dll(dll, n):
    current = dll['head']['next']
    count = 0
    while current != 'tail':
        if current > n:
            count += 1
        current = dll[current]['next']
    return count
```

Answer: > 10 → 14, 25, 11 → 3

🌲 Binary Tree — 8 exercises

1 MEDIUM

Sum of all node values

Write btc(tree, root) using BFS that sums all numeric node values. Answer for this tree.

GIVEN

```
tree = {
    'root': {'L': 8, 'R': 3},
    8:      {'L': 5, 'R': 9},
    3:      {'L': None, 'R': 4},
    5:      {'L': None, 'R': None},
    9:      {'L': None, 'R': None},
    4:      {'L': None, 'R': None}
}
# 'root' holds value 0 here (only children matter for the shape)
```

FULL SOLUTION

```
def btc(tree, root):
    queue = [root]
    total = 0
    while queue:
        current = queue.pop(0)
        lc = tree[current]['L']
        rc = tree[current]['R']
        if lc is not None:
            queue.append(lc)
        if rc is not None:
            queue.append(rc)
        if isinstance(current, (int, float)):
            total += current
    return total
```

Answer: 8 + 3 + 5 + 9 + 4 = **29** ('root' is not numeric, skipped)

2 MEDIUM

Count the leaves

Write count_leaves(tree, root) returning the number of leaf nodes (no L and no R child). Answer for this tree.

GIVEN

```
tree = {
    'root': {'L': 10, 'R': 20},
    10:     {'L': 6, 'R': 7},
    20:     {'L': None, 'R': 15},
    6:      {'L': None, 'R': None},
    7:      {'L': None, 'R': None},
    15:     {'L': None, 'R': None}
}
```

FULL SOLUTION

```
def count_leaves(tree, root):
    queue = [root]
    count = 0
    while queue:
        current = queue.pop(0)
        lc = tree[current]['L']
        rc = tree[current]['R']
        if lc is not None:
            queue.append(lc)
        if rc is not None:
            queue.append(rc)
        if lc is None and rc is None:
            count += 1
    return count
```

Answer: leaves = 6, 7, 15 → 3

3 HARD

Values between lo and hi

Write between(tree, root, lo, hi) returning a list of numeric values v with lo ≤ v ≤ hi. Answer for lo = 5, hi = 15.

GIVEN

```
tree = {
    'root': {'L': 12, 'R': 3},
    12:     {'L': 18, 'R': 7},
    3:      {'L': None, 'R': 20},
    18:     {'L': None, 'R': None},
    7:      {'L': None, 'R': None},
    20:     {'L': None, 'R': None}
}
lo, hi = 5, 15
```

FULL SOLUTION

```
def between(tree, root, lo, hi):
    queue = [root]
    result = []
    while queue:
        current = queue.pop(0)
        lc = tree[current]['L']
        rc = tree[current]['R']
        if lc is not None:
            queue.append(lc)
        if rc is not None:
            queue.append(rc)
        if isinstance(current, (int, float)):
            if lo <= current <= hi:
                result.append(current)
    return result
```

Answer: in [5,15]: 12, 7 → [12, 7] (order = BFS)

4 MEDIUM

Maximum node value

Write tree_max(tree, root) returning the largest numeric value via BFS. Answer for this tree.

GIVEN

```
tree = {
    'root': {'L': 14, 'R': 9},
    14:     {'L': 30, 'R': 2},
    9:      {'L': None, 'R': 25},
    30:     {'L': None, 'R': None},
    2:      {'L': None, 'R': None},
    25:     {'L': None, 'R': None}
}
```

FULL SOLUTION

```
def tree_max(tree, root):
    queue = [root]
    max_val = None
    while queue:
        current = queue.pop(0)
        lc = tree[current]['L']
        rc = tree[current]['R']
        if lc is not None:
            queue.append(lc)
        if rc is not None:
            queue.append(rc)
        if isinstance(current, (int, float)):
            if max_val is None or current > max_val:
                max_val = current
    return max_val
```

Answer: max(14,9,30,2,25) = 30

5 MEDIUM

Count all nodes

Write count_nodes(tree, root) counting every node reachable (including non-numeric root). Answer for this tree.

GIVEN

```
tree = {
    'root': {'L': 1, 'R': 2},
    1:      {'L': 3, 'R': 4},
    2:      {'L': 5, 'R': None},
    3:      {'L': None, 'R': None},
    4:      {'L': None, 'R': None},
    5:      {'L': None, 'R': None}
}
```

FULL SOLUTION

```
def count_nodes(tree, root):
    queue = [root]
    count = 0
    while queue:
        current = queue.pop(0)
        count += 1
        lc = tree[current]['L']
        rc = tree[current]['R']
        if lc is not None:
            queue.append(lc)
        if rc is not None:
            queue.append(rc)
    return count
```

Answer: root,1,2,3,4,5 → 6

6 HARD

Count even node values

Write count_even_tree(tree, root) returning how many numeric values are even. Answer for this tree.

GIVEN

```
tree = {
    'root': {'L': 4, 'R': 7},
    4:      {'L': 10, 'R': 5},
    7:      {'L': None, 'R': 8},
    10:     {'L': None, 'R': None},
    5:      {'L': None, 'R': None},
    8:      {'L': None, 'R': None}
}
```

FULL SOLUTION

```
def count_even_tree(tree, root):
    queue = [root]
    count = 0
    while queue:
        current = queue.pop(0)
        lc = tree[current]['L']
        rc = tree[current]['R']
        if lc is not None:
            queue.append(lc)
        if rc is not None:
            queue.append(rc)
        if isinstance(current, (int, float)) and current % 2 == 0:
            count += 1
    return count
```

Answer: even: 4, 10, 8 → 3

7 HARD

Does the tree contain a value?

Write contains(tree, root, target) returning True if target is a node value. Answer for target = 13.

GIVEN

```
tree = {
    'root': {'L': 6, 'R': 13},
    6:      {'L': 2, 'R': None},
    13:     {'L': 9, 'R': 21},
    2:      {'L': None, 'R': None},
    9:      {'L': None, 'R': None},
    21:     {'L': None, 'R': None}
}
target = 13
```

FULL SOLUTION

```
def contains(tree, root, target):
    queue = [root]
    while queue:
        current = queue.pop(0)
        if current == target:
            return True
        lc = tree[current]['L']
        rc = tree[current]['R']
        if lc is not None:
            queue.append(lc)
        if rc is not None:
            queue.append(rc)
    return False
```

Answer: 13 is the right child of root → True

8 HARD

Sum of leaf values only

Write sum_leaves(tree, root) that adds up the values of leaf nodes only. Answer for this tree.

GIVEN

```
tree = {
    'root': {'L': 5, 'R': 8},
    5:      {'L': 3, 'R': 6},
    8:      {'L': None, 'R': 10},
    3:      {'L': None, 'R': None},
    6:      {'L': None, 'R': None},
    10:     {'L': None, 'R': None}
}
```

FULL SOLUTION

```
def sum_leaves(tree, root):
    queue = [root]
    total = 0
    while queue:
        current = queue.pop(0)
        lc = tree[current]['L']
        rc = tree[current]['R']
        if lc is not None:
            queue.append(lc)
        if rc is not None:
            queue.append(rc)
        if lc is None and rc is None:
            if isinstance(current, (int, float)):
                total += current
    return total
```

Answer: leaves = 3, 6, 10 → 3 + 6 + 10 = 19

⛓ Singly Linked List — 8 exercises

1 MEDIUM

Sum of a singly linked list

Write sum_sll(llist) that adds all node values. The list ends at None. Answer for this list.

GIVEN

```
llist = {
    'head': 6,
    6: 11,
    11: 4,
    4: 9,
    9: None
}
```

FULL SOLUTION

```
def sum_sll(llist):
    current = llist['head']
    total = 0
    while current is not None:
        total += current
        current = llist[current]
    return total
```

Answer: 6 + 11 + 4 + 9 = 30

2 MEDIUM

Length of the list

Write count_sll(llist) returning the number of nodes (not counting 'head'). Answer for this list.

GIVEN

```
llist = {
    'head': 3,
    3: 8,
    8: 1,
    1: 7,
    7: 5,
    5: None
}
```

FULL SOLUTION

```
def count_sll(llist):
    current = llist['head']
    count = 0
    while current is not None:
        count += 1
        current = llist[current]
    return count
```

Answer: nodes = 3,8,1,7,5 → 5

3 MEDIUM

Maximum value

Write max_sll(llist) returning the largest node value. Answer for this list.

GIVEN

```
llist = {
    'head': 12,
    12: 7,
    7: 19,
    19: 4,
    4: 15,
    15: None
}
```

FULL SOLUTION

```
def max_sll(llist):
    current = llist['head']
    max_val = current
    while current is not None:
        if current > max_val:
            max_val = current
        current = llist[current]
    return max_val
```

Answer: max(12,7,19,4,15) = 19

4 HARD

Does it contain a value?

Write contains_sll(llist, target) returning True/False. Answer for target = 14.

GIVEN

```
llist = {
    'head': 2,
    2: 14,
    14: 6,
    6: 8,
    8: None
}
target = 14
```

FULL SOLUTION

```
def contains_sll(llist, target):
    current = llist['head']
    while current is not None:
        if current == target:
            return True
        current = llist[current]
    return False
```

Answer: 14 is the 2nd node → True

5 HARD

Add a node at the front (trace)

Write add_front(llist, node) that inserts node before the head. Apply add_front(llist, 99) and give the new order.

GIVEN

```
llist = {
    'head': 5,
    5: 8,
    8: 3,
    3: None
}
# call: add_front(llist, 99)
```

FULL SOLUTION

```
def add_front(llist, node):
    llist[node] = llist['head']
    llist['head'] = node
    return llist
```

Answer: 99 now points to old head 5 → 99 → 5 → 8 → 3

6 HARD

Remove the first node (trace)

Write remove_first(llist) that deletes the first node after head. Apply it and give the new order.

GIVEN

```
llist = {
    'head': 10,
    10: 20,
    20: 30,
    30: None
}
```

FULL SOLUTION

```
def remove_first(llist):
    first = llist['head']
    llist['head'] = llist[first]
    del llist[first]
    return llist
```

Answer: head moves to 20, node 10 deleted → **20 → 30**

7 MEDIUM

Count even values

Write count_even_sll(llist) returning how many node values are even. Answer for this list.

GIVEN

```
llist = {
    'head': 4,
    4: 7,
    7: 10,
    10: 3,
    3: 6,
    6: None
}
```

FULL SOLUTION

```
def count_even_sll(llist):
    current = llist['head']
    count = 0
    while current is not None:
        if current % 2 == 0:
            count += 1
        current = llist[current]
    return count
```

Answer: even = 4, 10, 6 → 3

8 HARD

Average of the list

Write average_sll(llist) returning the mean of all values. Answer for this list.

GIVEN

```
llist = {
    'head': 8,
    8: 12,
    12: 4,
    4: 16,
    16: None
}
```

FULL SOLUTION

```
def average_sll(llist):
    current = llist['head']
    total = 0
    count = 0
    while current is not None:
        total += current
        count += 1
        current = llist[current]
    return total / count
```

Answer: (8+12+4+16)/4 = 40/4 = 10.0

📦 Dictionary — 8 exercises

1 MEDIUM

Letter frequency

Write count_letters(text) returning a dict mapping each letter to its count (lowercased). Give the dict for the word below.

GIVEN

```
text = "banana"
```

FULL SOLUTION

```
def count_letters(text):
    d = {}
    text = text.lower()
    for x in text:
        if x in d:
            d[x] += 1
        else:
            d[x] = 1
    return d
```

Answer: **{'b': 1, 'a': 3, 'n': 2}**

2 MEDIUM

Word lengths

Write word_lengths(text) mapping each word to its length. Give the dict for the sentence below.

GIVEN

```
text = "the quick brown fox"
```

FULL SOLUTION

```
def word_lengths(text):
    d = {}
    words = text.split()
    for w in words:
        d[w] = len(w)
    return d
```

Answer: **{'the': 3, 'quick': 5, 'brown': 5, 'fox': 3}**

3 HARD

Query grades by subject and score

Each student maps to (subject, score). Write query_grades(grades, subject, min_score) returning two lists: students of that subject, and students scoring above min_score. Answer for subject='Math', min_score=25.

GIVEN

```
grades = {
    'Anna':  ('Math', 28),
    'Bob':   ('Physics', 30),
    'Carla': ('Math', 22),
    'Dino':  ('Math', 26),
    'Elena': ('Physics', 19)
}
subject, min_score = 'Math', 25
```

FULL SOLUTION

```
def query_grades(grades, subject, min_score):
    by_subject = []
    high_scorers = []
    for name, info in grades.items():
        subj = info[0]
        score = info[1]
        if subj == subject:
            by_subject += [name]
        if score > min_score:
            high_scorers += [name]
    return by_subject, high_scorers
```

Answer: by_subject(Math) = ['Anna','Carla','Dino'] · high_scorers(>25) = ['Anna','Bob','Dino'] → (['Anna','Carla','Dino'], ['Anna','Bob','Dino'])

4 HARD

Query a music catalog

Each song maps to (artist, duration). Write query_catalog(catalog, artist, min_dur) returning (songs by that artist, songs longer than min_dur). Answer for artist='Mina', min_dur=180.

GIVEN

```
catalog = {
    'Song1': ('Mina', 200),
    'Song2': ('Celentano', 150),
    'Song3': ('Mina', 175),
    'Song4': ('Mina', 240),
    'Song5': ('Battisti', 190)
}
artist, min_dur = 'Mina', 180
```

FULL SOLUTION

```
def query_catalog(catalog, artist, min_dur):
    by_artist = []
    long_songs = []
    for title, info in catalog.items():
        art = info[0]
        dur = info[1]
        if art == artist:
            by_artist += [title]
        if dur > min_dur:
            long_songs += [title]
    return by_artist, long_songs
```

Answer: by_artist(Mina)=['Song1','Song3','Song4'] · long(>180)=['Song1','Song4','Song5'] → **(['Song1','Song3','Song4'], ['Song1','Song4','Song5'])**

5 HARD

Invert and group a dictionary

Write invert_and_group(d) that builds a new dict value → list of keys having that value. Give the result for the dict below.

GIVEN

```
d = {
    'a': 1,
    'b': 2,
    'c': 1,
    'd': 3,
    'e': 2
}
```

FULL SOLUTION

```
def invert_and_group(d):
    inverse = {}
    for k in d:
        val = d[k]
        if val not in inverse:
            inverse[val] = [k]
        else:
            inverse[val].append(k)
    return inverse
```

Answer: **{1: ['a','c'], 2: ['b','e'], 3: ['d']}**

6 MEDIUM

Filter a phonebook by type

Each contact maps to (number, type). Write by_type(pb, contact_type) returning names matching the type. Answer for contact_type='work'.

GIVEN

```
pb = {
    'Marco': (3331111, 'home'),
    'Lucia': (3332222, 'work'),
    'Gino':  (3333333, 'work'),
    'Sara':  (3334444, 'home')
}
contact_type = 'work'
```

FULL SOLUTION

```
def by_type(pb, contact_type):
    result = []
    for name in pb:
        if pb[name][1] == contact_type:
            result += [name]
    return result
```

Answer: work contacts → **['Lucia', 'Gino']**

7 HARD

Key with the maximum value

Write max_key(d) returning the key whose value is largest. Answer for the dict below.

GIVEN

```
d = {
    'apple': 12,
    'pear': 7,
    'kiwi': 19,
    'plum': 15
}
```

FULL SOLUTION

```
def max_key(d):
    best_key = None
    best_val = None
    for k in d:
        if best_val is None or d[k] > best_val:
            best_val = d[k]
            best_key = k
    return best_key
```

Answer: largest value 19 → 'kiwi'

8 HARD

Build a frequency dict from a list

Write frequency(lst) returning a dict element → how many times it appears. Give the result for the list below.

GIVEN

```
lst = [4, 7, 4, 4, 9, 7, 2]
```

FULL SOLUTION

```
def frequency(lst):
    d = {}
    for x in lst:
        if x in d:
            d[x] += 1
        else:
            d[x] = 1
    return d
```

Answer: {4: 3, 7: 2, 9: 1, 2: 1}